Quantum Portfolio Optimization with QAOA: A Six-Asset Statevector Study with Constraint-Aware Classical Repair

Shikher Singh *Qiskit Fall Fest 2026 × IIIT Delhi, Track 2: Quantum Optimization* Repository: https://github.com/Techz-Shikher/qiskit-quantum-portfolio-optimization

Abstract

Cardinality-constrained portfolio selection is a binary quadratic optimization problem that maps naturally onto the Quantum Approximate Optimization Algorithm (QAOA). This paper studies a six-asset synthetic instance in which exactly three assets must be chosen, using an ideal statevector simulation of QAOA at depths p = 1 to 4, validated against an exhaustive classical search. We report how circuit depth and the constraint-penalty weight λ affect expected cost, feasibility probability and the probability of sampling the exact optimum, and we introduce Q-Repair, a classical post-processing step that maps each sampled bitstring to a nearby feasible portfolio. At p = 3 the circuit places 92.69% of its probability mass on feasible portfolios but only 5.03% on the exact optimum; Q-Repair raises the mapped optimum mass to 8.81%. Results are non-monotonic in depth and penalty strength, optimizer runs exhausted their evaluation budget, and no hardware execution or quantum advantage is claimed.

Index Terms: QAOA, portfolio optimization, QUBO, penalty methods, constraint handling, statevector simulation, Qiskit.

I. Introduction

Portfolio selection under a cardinality constraint, in the tradition of Markowitz's mean-variance model [1], is combinatorial: choosing K of n assets yields C(n, K) candidates. Encoding the selection as binary variables turns the task into a quadratic unconstrained binary optimization (QUBO) problem [2], the natural input to variational quantum algorithms such as QAOA [3]. Prior work has benchmarked QAOA on portfolio problems [4]; the present study is deliberately small and fully reproducible, so that every QAOA result can be compared against the exact optimum.

The contributions of this paper are:

1. An exact classical reference for a six-asset, K = 3 synthetic instance, generated deterministically from a fixed NumPy seed.
2. A depth sweep (p = 1 to 4) and a penalty sweep (eight values of λ) reporting expected cost, feasibility and optimum probability.
3. Q-Repair, a classical, constraint-aware post-processing step, with an explicit statement of what it does and does not demonstrate.
4. A separate 12-asset classical baseline, included for scale context only.

II. Problem Formulation

Let $x_i \in \{0, 1\}$ indicate whether asset i is selected, for $i = 1, \ldots, n$. With covariance matrix Σ, expected-return vector μ, risk-aversion coefficient q and penalty weight λ, the penalized objective to minimize is

> $$C(x) = q\, x^{T}\Sigma x - \mu^{T}x + \lambda\,(\Sigma_i x_i - K)^2$$

subject to the intended constraint $\Sigma_i x_i = K$. The first term penalizes risk, the second rewards expected return, and the third penalizes portfolios whose size differs from K. In this study n = 6, K = 3 and q = 0.5. Lower cost is better. A portfolio is called *feasible* if it contains exactly K assets.

III. Data and Reproducibility

The instance is synthetic and generated deterministically:

```
rng = np.random.default_rng(2026)
n, K, q = 6, 3, 0.5
mu = rng.uniform(0.05, 0.25, n)
A = rng.normal(0, 0.1, (n, n))
sigma = A @ A.T
```

Because $\Sigma = AA^{T}$, the covariance matrix is positive semidefinite. The instance is a benchmark, not a statement about real market behavior.

IV. Methods

A. Exact Classical Reference

All $2^6 = 64$ bitstrings are enumerated. Of these, $C(6, 3) = 20$ satisfy the cardinality constraint. The lowest-cost feasible portfolio is the reference optimum against which QAOA is judged.

B. QAOA Statevector Simulation

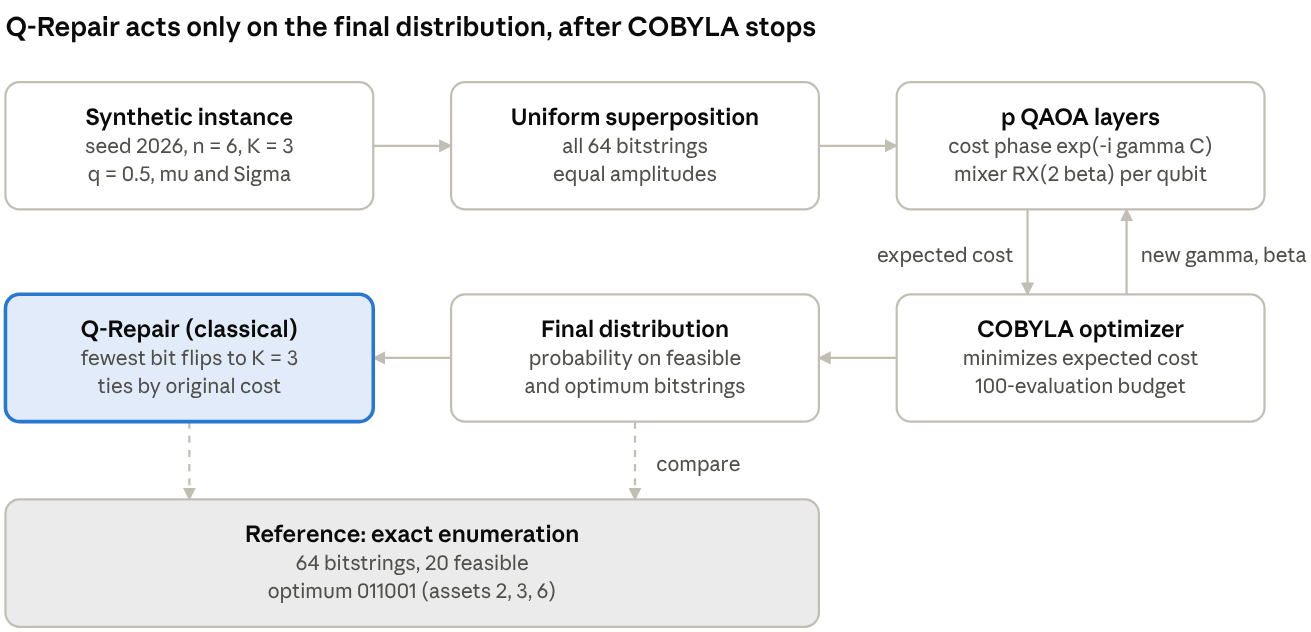
The state is initialized as a uniform superposition over all 64 bitstrings. Each of p layers applies (i) a cost phase $\exp(-i\gamma C(x))$ to every basis-state amplitude and (ii) a mixer $RX(2\beta)$ on each of the six qubits. Parameters (γ, β) are optimized with COBYLA to minimize the expected penalized cost. The simulation applies the cost phases and single-qubit mixers directly to a statevector; it is not a hardware execution and not a GPU-accelerated run.

C. Experimental Protocols

The *depth sweep* evaluates $p = 1, 2, 3, 4$. The *penalty sweep* evaluates $\lambda \in \{0.01, 0.05, 0.1, 0.25, 0.5, 1, 2, 5\}$ at $p = 1$. For each run we record the expected penalized cost, the total probability on feasible bitstrings, and the probability of the exact optimum.

D. Q-Repair

Q-Repair maps every sampled bitstring to a feasible portfolio by changing the fewest bits needed to reach exactly K selected assets, resolving ties by the original portfolio objective. The probability of each raw bitstring is then moved to its repaired portfolio. Q-Repair is purely classical post-processing.



method pipeline · 6 steps + exact reference

Pipeline overview. COBYLA and the QAOA layers loop until the evaluation budget is spent; Q-Repair then acts once on the final distribution, and results are compared with exact enumeration.

V. Results

A. Classical Optimum

Quantity	Value
Bitstrings enumerated	64
Feasible portfolios	20
Optimal assets (one-based)	2, 3, 6
Optimal bitstring	011001
Optimal objective	−0.503015

B. Effect of Circuit Depth

p	Expected cost	Feasible probability	Optimum probability
1	0.110713	57.90%	3.68%
2	0.202474	67.94%	3.38%
3	−0.101954	92.69%	5.03%
4	−0.213192	88.23%	4.55%

Depth p = 3 gives the highest feasibility (92.69%) and the highest optimum probability (5.03%), whereas p = 4 gives the lowest expected cost (−0.213192). Expected cost, feasibility and optimum probability therefore do not peak at the same depth, and none improves monotonically with p. All runs at p = 2 to 4 reached the 100-evaluation budget.

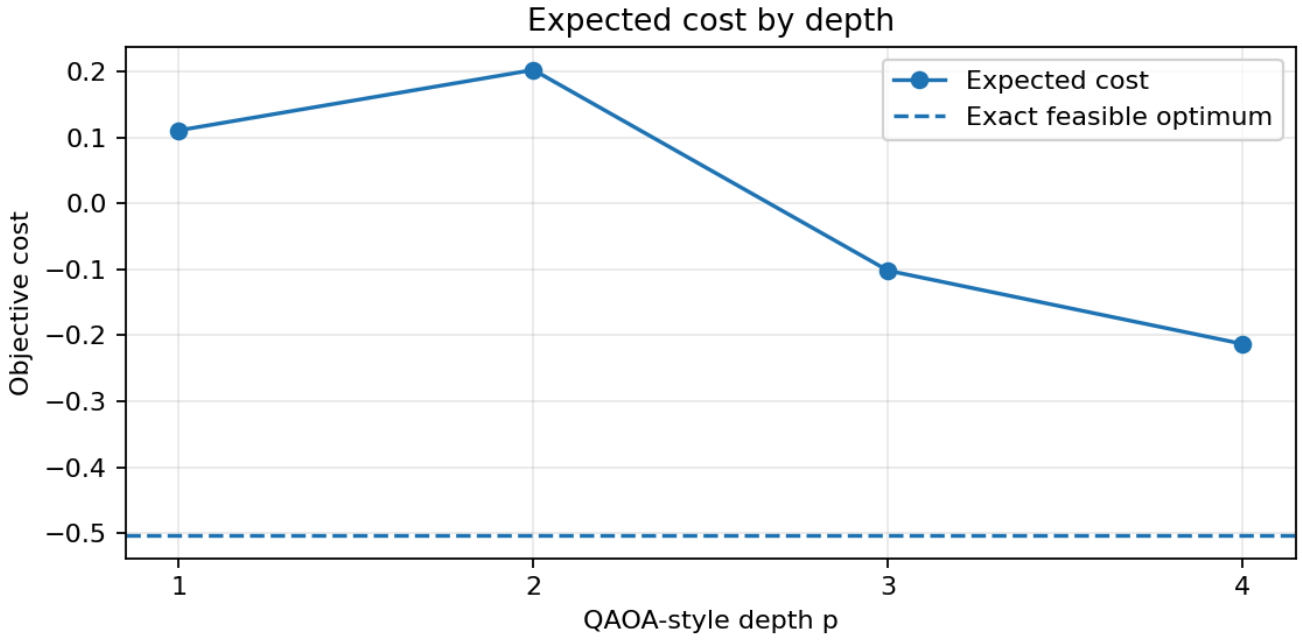


Fig. 1. Expected penalized cost versus depth p. The dashed line marks the exact feasible optimum (−0.503015); no depth reaches it.

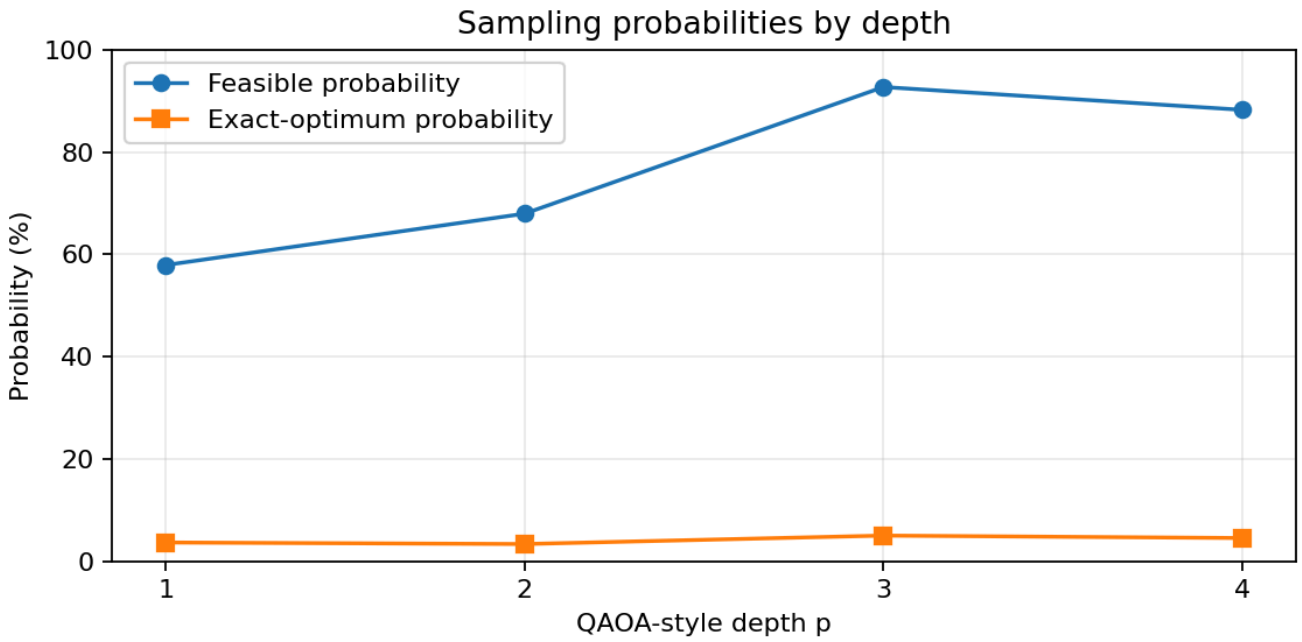


Fig. 2. Feasible and exact-optimum probability versus depth p.

C. Effect of Penalty Strength

λ	Feasible probability	Optimum probability
0.01	3.8831%	1.4133%
0.05	26.3371%	2.1698%
0.1	47.3786%	2.9138%
0.25	62.3480%	2.7462%
0.5	66.1235%	3.4483%
1	57.9032%	3.6819%
2	67.0550%	3.2974%
5	67.1011%	3.3694%

Feasibility rises steeply as λ increases from 0.01 to about 0.25 and then saturates near 67%, with a dip at λ = 1. The highest feasibility (67.10%) occurs at λ = 5, while the highest optimum probability (3.68%) occurs at λ = 1. Feasibility and optimum probability need not peak together. Some optimizer runs returned success = False, so these values are observed runs rather than globally optimized penalty settings.

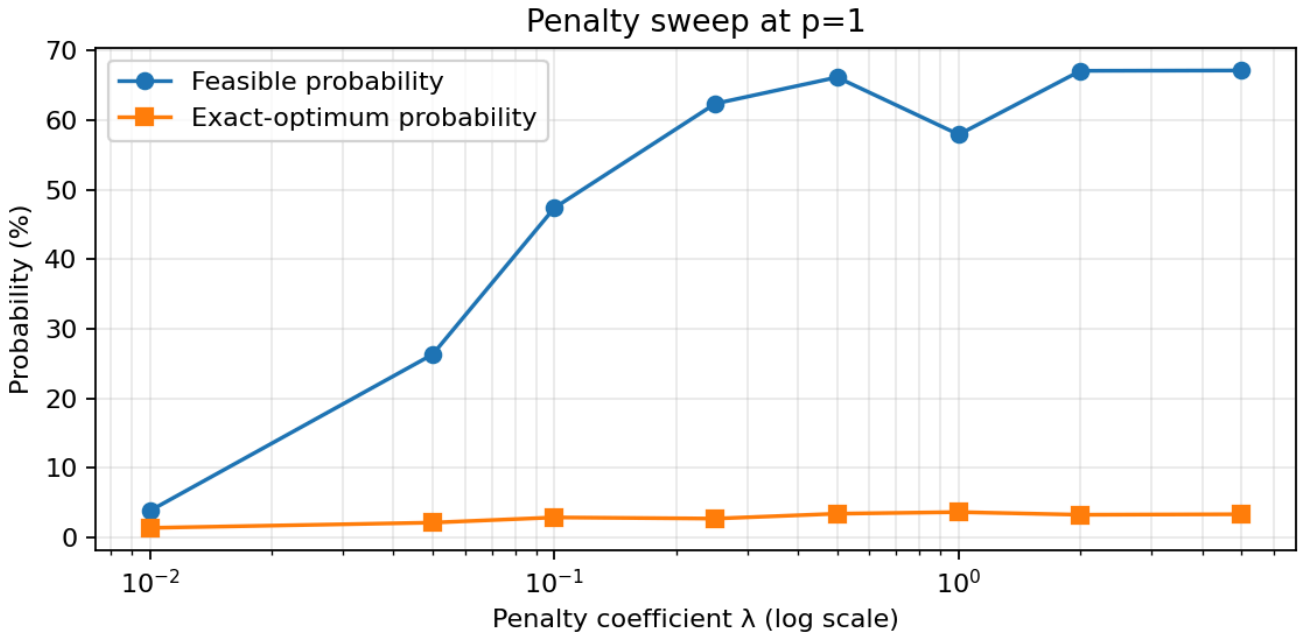


Fig. 3. Penalty sweep at p = 1 (λ on a log scale).

D. Q-Repair

Applied to the selected p = 3 run:

Metric	Raw QAOA	After Q-Repair
Probability on exact optimum	5.03%	8.81%
Expected cost (original objective)	−0.382748 (conditional on feasibility)	−0.389191

Q-Repair increases the mapped optimum mass by 3.78 percentage points, about a 1.75-fold increase. All eight optimizer starts reported success = False and reached the evaluation limit, so the underlying QAOA distribution is itself an unconverged result.

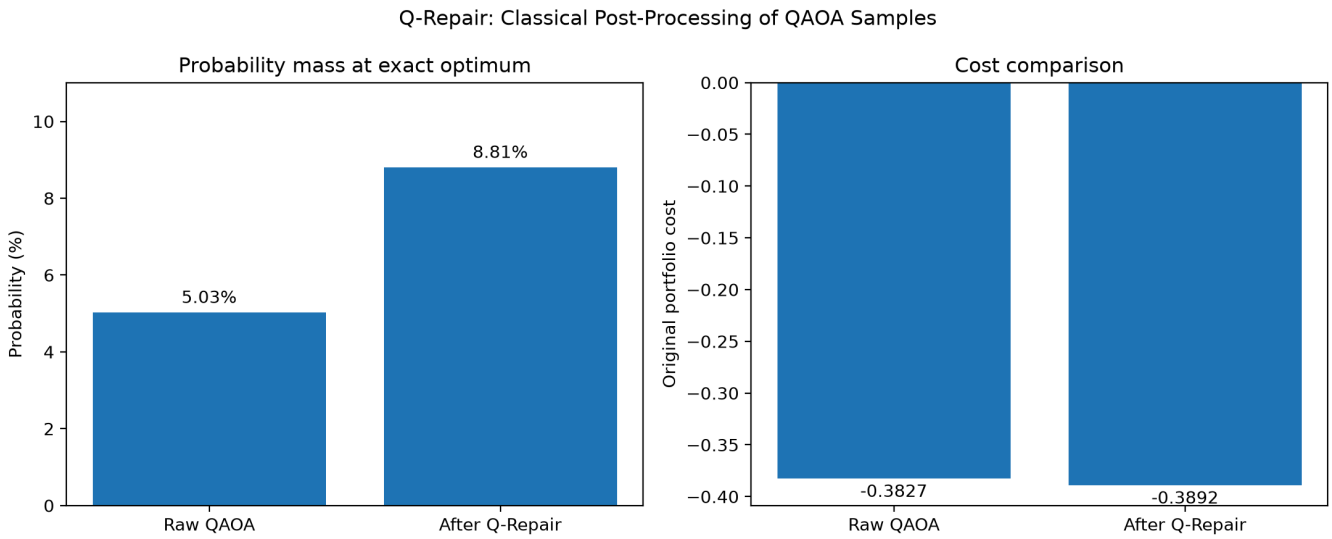


Fig. 4. Q-Repair on the selected p = 3 run: optimum probability mass 5.03% to 8.81%, original-objective cost −0.3827 to −0.3892.

E. Twelve-Asset Classical Baseline

For n = 12 and K = 6 there are C(12, 6) = 924 feasible portfolios. Exhaustive enumeration yields the optimal set (2, 6, 7, 9, 11, 12) with objective −1.066392. This is a classical result only; no 12-asset QAOA experiment was completed.

VI. Discussion

The results illustrate three points. First, a high feasibility probability does not imply a high probability of the optimum: at p = 3 the circuit is feasible 92.69% of the time yet finds the optimum only 5.03% of the time, close to what uniform sampling over 20 feasible portfolios would give (5%). Second, penalty weight acts as a trade-off between constraint satisfaction and objective fidelity, and the observed behavior is non-monotonic. Third, Q-Repair shows that cheap classical constraint handling can shift probability mass toward the optimum, but because the repair is classical, this result must not be read as quantum advantage.

Limitations

- The data are synthetic and the instance is a single seed, so conclusions are instance-specific.
- Simulation is ideal statevector: no noise, no hardware run, no gate-level circuit comparison.

- COBYLA was limited to 100 evaluations, and runs at p = 2 to 4 hit the limit; sensitivity to initialization remains significant.
- The penalty sweep covers only eight values at p = 1.
- Six assets are exactly solvable by brute force, so the classical baseline is trivially stronger than QAOA here.

VII. Conclusion and Future Work

This study provides a reproducible benchmark of QAOA for cardinality-constrained portfolio selection on a six-asset synthetic instance. Depth and penalty strength influence feasibility and solution quality differently, and classical Q-Repair raised mapped optimum probability from 5.03% to 8.81% in the selected run. No claim of quantum advantage or hardware execution is made. Future work includes improved optimizer convergence, repeated seeds with confidence intervals, a gate-level Qiskit circuit comparison, stronger classical baselines, historical financial data, and QAOA on the 12-asset instance.

Statement on AI Tool Use

ChatGPT (OpenAI) was used for brainstorming, concept explanations, debugging support, documentation, and report and presentation structure. Suggestions were reviewed and adapted; the author is responsible for validating the implementation and results.

References

[1] H. Markowitz, "Portfolio selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.

[2] A. Lucas, "Ising formulations of many NP problems," *Frontiers in Physics*, vol. 2, art. 5, 2014.

[3] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," arXiv:1411.4028, 2014.

[4] S. Brandhofer et al., "Benchmarking the performance of portfolio optimization with QAOA," *Quantum Machine Intelligence*, vol. 5, 2023.

[5] M. J. D. Powell, "A direct search optimization method that models the objective and constraint functions by linear interpolation," in *Advances in Optimization and Numerical Analysis*, Kluwer, 1994, pp. 51–67.

[6] Project repository: https://github.com/Techz-Shikher/qiskit-quantum-portfolio-optimization

[7] Challenge repository: https://github.com/SnehilK3372/qiskit-fall-fest-2026-IIITD

[8] Challenge tutorial notebook: https://colab.research.google.com/github/SnehilK3372/qiskit-fall-fest-2026-IIITD/blob/main/hackathon/track-2-quantum-optimization/challenge_2_tutorial.ipynb